

Introducción



Fernando Tricas García – Javier Campos Laclaustra

Universidad de Zaragoza

Curso 2025/26

Presentación adaptada de la original de J. Campos

Este documento está sujeto a una licencia de uso Creative Commons. No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original.



Resumen



- 1 Motivación
- 2 Un ejemplo
- 3 ¿Todavía?
- 4 Algunas ideas
- 5 Bibliografía



Una definición

Algoritmia

Tratamiento sistemático de técnicas fundamentales para el diseño y análisis de algoritmos eficientes



¿Hace falta?

- Computadores cada vez más rápidos y a más bajo precio:
 - Se resuelven problemas de cálculo antes impensables.
 - Inconscientemente se tiende a restar importancia al concepto de eficiencia.
- Existen problemas que seguirán siendo intratables si se aplican ciertos algoritmos por mucho que se aceleren los computadores

⇒ importancia de nuevas soluciones eficientes



Ejemplo

En Agosto de 1977, Scientific American proponía a sus lectores el reto consistente en descifrar un mensaje secreto, para así ganar cien dólares. Parecía algo seguro: se estimaba en aquel momento que el ordenador más rápido existente, empleando el algoritmo más eficiente de los conocidos, no podría ganar la apuesta salvo funcionando sin interrupción durante un tiempo equivalente a millones de veces la edad del Universo. Sin embargo, ocho meses de cálculo que comenzaron dieciséis años después bastaron para la tarea. ¿Qué había pasado?...

*G. Brassard y P. Bratley
Fundamentos de Algoritmia (Prólogo)*



La nube al revés

The “RSA challenge” published in the August 1977 issue of Scientific American (in Martin Gardner’s column) is still open, and the \$100 prize offer still stands. This prize can be won by factoring the RSA modulus published there, which is:

RSA-129 =

1143816257578888676692357799761466120102182
9672124236256256184293570693524573389783059
7123563958705058989075147599290026879543541

Noviembre 1993

https://en.wikipedia.org/wiki/RSA_Factoring_Challenge

<https://web.archive.org/web/20100419054922/https://www.rsa.com/rsalabs/node.asp?id=2094>

‘Martin Gardner, Scientific American. Mathematical Games, August 1977. A new kind of cipher that would take millions of years to break’ (de pago). <https://www.scientificamerican.com/article/mathematical-games-1977-08>

<https://www.math.upenn.edu/~kazdan/210S19/Notes/crypto/Gardner-RSA-1977.pdf>



Primeros intentos

- Rivest estimaba 4×10^{16} años de computación para lograrlo
 - RSA-129 se factorizó el 2 de abril de 1994 (criba cuadrática)
 - Cálculo distribuido (e-mail y FTP)
 - Código fuente
 - 'That said, it runs happily on any Unix box with at least 8Mb of physical memory.'
 - Unos 600 computadores



Por si alguien quiere comprobarlo

$$\begin{array}{r}
 1143816257578888676692357799761466120102182 \\
 9672124236256256184293570693524573389783059 \\
 7123563958705058989075147599290026879543541 = \\
 \\
 34905295108476509491478496199038 \\
 98133417764638493387843990820577 \\
 \times \\
 32769132993266709549961988190834 \\
 461413177642967992942539798288533 \\
 \hline
 \end{array}$$

http://www.revistasic.com/revista40/agorarevista_40.htm

<http://en.wikipedia.org/wiki/RSA-129>

Para leer la historia:

<http://unaaldia.hispasec.com/2013/10/martin-gardner-rsa-y-otros-pasatiempos.html>



Lo que no es

- Un curso de programación (ya deberíais saber programar)
- Un curso de estructuras de datos (ya deberíais conocer las fundamentales)

TAMPOCO ES

- una colección de “recetas” o algoritmos listos para ser introducidos en el computador para resolver problemas específicos

Si tu problema es ordenar un fichero secuencial de enteros entonces ejecuta el algoritmo A026.



¿Entonces? Objetivo principal

- dar más herramientas fundamentales para facilitar el desarrollo de programas

¿qué herramientas?:

técnicas o “esquemas” de diseño de algoritmos eficientes

Metodología



- Presentar cada esquema de forma genérica, incidiendo en los principios que conducen a él,
- Ilustrar el esquema mediante ejemplos concretos de algoritmos tomados de varios dominios de aplicación



Un ejemplo sencillo

Multiplicación de dos enteros positivos con lápiz y papel.

En España

$$\begin{array}{r}
 9\ 8\ 1 \\
 1\ 2\ 3\ 4 \\
 \hline
 3\ 9\ 2\ 4 \\
 2\ 9\ 4\ 3 \\
 1\ 9\ 6\ 2 \\
 9\ 8\ 1 \\
 \hline
 1\ 2\ 1\ 0\ 5\ 5\ 4
 \end{array}$$

En Inglaterra

$$\begin{array}{r}
 9\ 8\ 1 \\
 1\ 2\ 3\ 4 \\
 \hline
 9\ 8\ 1 \\
 1\ 9\ 6\ 2 \\
 2\ 9\ 4\ 3 \\
 3\ 9\ 2\ 4 \\
 \hline
 1\ 2\ 1\ 0\ 5\ 5\ 4
 \end{array}$$

Ambos métodos son muy similares, los llamaremos algoritmo “clásico” de multiplicación.



Algoritmo de multiplicación 'a la rusa'

→	981	1234	1234
	490	2468	
→	245	4936	4936
	122	9872	
→	61	19744	19744
	30	39488	
→	15	78976	78976
→	7	157952	157952
→	3	315904	315904
→	1	631808	631808
			1210554

Ventajas:

- No hay que almacenar los productos parciales.
- Sólo hay que saber sumar y dividir por 2.



Otro algoritmo de multiplicación

De momento, exigimos que ambos números tengan igual n^o de cifras y que éste sea potencia de 2.

Por ejemplo: 0981 y 1234



Otro algoritmo de multiplicación

De momento, exigimos que ambos números tengan igual n^o de cifras y que éste sea potencia de 2.

Por ejemplo: 0981 y 1234

En primer lugar, partimos ambos números por la mitad y hacemos cuatro productos:

09	81
12	34

	multiplicar	desplazar	resultado
1)	09 12	4	108
2)	09 34	2	306 . .
3)	81 12	2	972 . .
4)	81 34	0	2754
			1210554



Otro algoritmo de multiplicación

De momento, exigimos que ambos números tengan igual n^o de cifras y que éste sea potencia de 2.

Por ejemplo: 0981 y 1234

	multiplicar	desplazar	resultado
1)	09 12	4	108
2)	09 34	2	306 . .
3)	81 12	2	972 . .
4)	81 34	0	2754
			1210554

Los desplazamientos

- 4: doble del n^o de cifras
- 2: n^o de cifras

Es decir, hemos reducido un producto de números de 4 cifras en cuatro productos de números de 2 cifras, varios desplazamientos y una suma.



Otro algoritmo de multiplicación

Ahora, los productos de números de 2 cifras pueden hacerse con la misma técnica.

Por ejemplo, 09 y 12:

	multiplicar	desplazar	resultado
1)	0 1	2	0 . .
2)	0 2	1	0 .
3)	9 1	1	9 .
4)	9 2	0	18
			108

- Es un ejemplo de algoritmo que utiliza la técnica de “divide y vencerás” .



Otro algoritmo de multiplicación

Ahora, los productos de números de 2 cifras pueden hacerse con la misma técnica.

Por ejemplo, 09 y 12:

	multiplicar	desplazar	resultado
1)	0 1	2	0 . .
2)	0 2	1	0 .
3)	9 1	1	9 .
4)	9 2	0	18
			108

- Es un ejemplo de algoritmo que utiliza la técnica de “divide y vencerás” .
- Tal y como lo hemos presentado. . .

NO mejora en eficiencia al algoritmo clásico.



Otro algoritmo de multiplicación

- Pero, puede mejorarse:

Es posible reducir un producto de dos números de muchas cifras a:

- 3 (en vez de 4) productos de números de la mitad de cifras,

y éste SÍ que mejora al algoritmo clásico.

- Y aún se conocen métodos más rápidos para multiplicar números muy grandes.

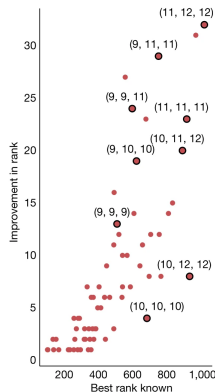


Se sigue investigando

- A. Fawzi, et al. 'Discovering faster matrix multiplication algorithms with reinforcement learning' Nature, 5 October 2022.

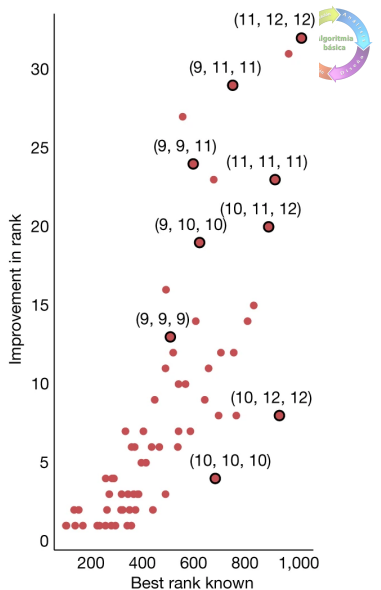
<https://www.nature.com/articles/s41586-022-05172-4>.

Size (n, m, p)	Best method known	Best rank known	AlphaTensor rank Modular Standard
(2, 2, 2)	(Strassen, 1969) ²	7	7
(3, 3, 3)	(Laderman, 1976) ¹⁵	23	23
(4, 4, 4)	(Strassen, 1969) ² (2, 2, 2) $\otimes$ (2, 2, 2)	49	47
(5, 5, 5)	(3, 5, 5) + (2, 5, 5)	98	96
(2, 2, 3)	(2, 2, 2) + (2, 2, 1)	11	11
(2, 2, 4)	(2, 2, 2) + (2, 2, 2)	14	14
(2, 2, 5)	(2, 2, 2) + (2, 2, 3)	18	18
(2, 3, 3)	(Hopcroft and Kerr, 1971) ¹⁶	15	15
(2, 3, 4)	(Hopcroft and Kerr, 1971) ¹⁶	20	20
(2, 3, 5)	(Hopcroft and Kerr, 1971) ¹⁶	25	25
(2, 4, 4)	(Hopcroft and Kerr, 1971) ¹⁶	26	26
(2, 4, 5)	(Hopcroft and Kerr, 1971) ¹⁶	33	33
(2, 5, 5)	(Hopcroft and Kerr, 1971) ¹⁶	40	40
(3, 3, 4)	(Smirnov, 2013) ¹⁸	29	29
(3, 3, 5)	(Smirnov, 2013) ¹⁸	36	36
(3, 4, 4)	(Smirnov, 2013) ¹⁸	38	38
(3, 4, 5)	(Smirnov, 2013) ¹⁸	48	47
(3, 5, 5)	(Sedoglavic and Smirnov, 2021) ¹⁹	58	58
(4, 4, 5)	(4, 4, 2) + (4, 4, 3)	64	63
(4, 5, 5)	(2, 5, 5) $\otimes$ (2, 1, 1)	80	76



Se

Size (n, m, p)	Best method known	Best rank known	AlphaTensor rank Modular Standard	
(2, 2, 2)	(Strassen, 1969) ²	7	7	7
(3, 3, 3)	(Laderman, 1976) ¹⁵	23	23	23
(4, 4, 4)	(Strassen, 1969) ² (2, 2, 2) $\otimes$ (2, 2, 2)	49	47	49
(5, 5, 5)	(3, 5, 5) + (2, 5, 5)	98	96	98
(2, 2, 3)	(2, 2, 2) + (2, 2, 1)	11	11	11
(2, 2, 4)	(2, 2, 2) + (2, 2, 2)	14	14	14
(2, 2, 5)	(2, 2, 2) + (2, 2, 3)	18	18	18
(2, 3, 3)	(Hopcroft and Kerr, 1971) ¹⁶	15	15	15
(2, 3, 4)	(Hopcroft and Kerr, 1971) ¹⁶	20	20	20
(2, 3, 5)	(Hopcroft and Kerr, 1971) ¹⁶	25	25	25
(2, 4, 4)	(Hopcroft and Kerr, 1971) ¹⁶	26	26	26
(2, 4, 5)	(Hopcroft and Kerr, 1971) ¹⁶	33	33	33
(2, 5, 5)	(Hopcroft and Kerr, 1971) ¹⁶	40	40	40
(3, 3, 4)	(Smirnov, 2013) ¹⁸	29	29	29
(3, 3, 5)	(Smirnov, 2013) ¹⁸	36	36	36
(3, 4, 4)	(Smirnov, 2013) ¹⁸	38	38	38
(3, 4, 5)	(Smirnov, 2013) ¹⁸	48	47	47
(3, 5, 5)	(Sedoglavic and Smirnov, 2021) ¹⁹	58	58	58
(4, 4, 5)	(4, 4, 2) + (4, 4, 3)	64	63	63
(4, 5, 5)	(2, 5, 5) $\otimes$ (2, 1, 1)	80	76	76





Se sigue investigando

- A. Fawzi, et al. 'Discovering faster matrix multiplication algorithms with reinforcement learning' Nature, 5 October 2022.
<https://www.nature.com/articles/s41586-022-05172-4>.
- 'When AI "Played" Math, It Cracked an Internet Chokepoint' IEEE Spectrum 26 Oct 2022.
<https://spectrum.ieee.org/matrix-multiplication-deepmind>

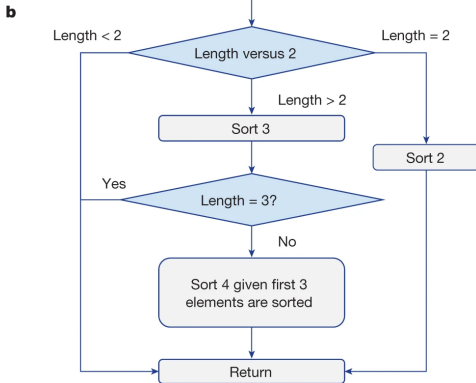
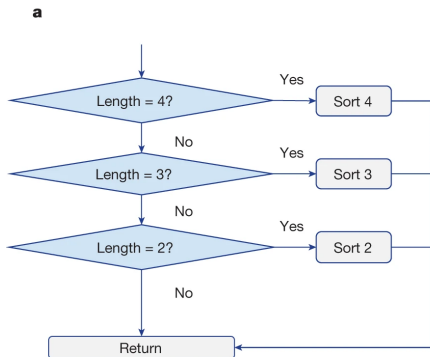
"For example, AlphaTensor discovered an algorithm for multiplying four-by-four matrices in just 47 steps, improving on Strassen's 50-year-old algorithm, which uses 49."



Se sigue investigando

D.J. Makowitz, et al. 'Faster sorting algorithms discovered using deep reinforcement learning'. Nature 7 June 2023.

<https://www.nature.com/articles/s41586-023-06004-9>





Otro algoritmo de multiplicación más

Método chino

Multiplicar 21 por 31.

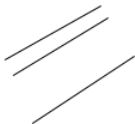


Otro algoritmo de multiplicación más

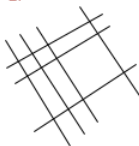
Método chino

Multiplicar 21 por 31.

1.



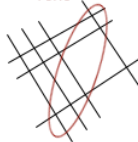
2.



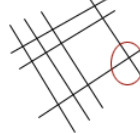
Hundreds



Tens



Unes



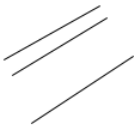


Otro algoritmo de multiplicación más

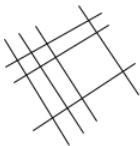
Método chino

Multiplicar 21 por 31.

1.



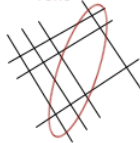
2.



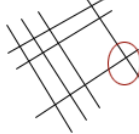
Hundreds



Tens



Unes



651



Ideas clave

- Incluso para un problema tan básico pueden construirse MUCHAS soluciones.
- El método clásico lo usamos con lápiz y papel porque nos resulta muy familiar (lo que se aprende en la infancia...).
- El método “a la rusa” se implementa en hardware en los computadores por la naturaleza elemental de los cálculos intermedios.
- El método de divide y vencerás es más rápido si se quiere multiplicar números grandes.



Ideas clave

La algoritmia estudia las propiedades de los algoritmos y nos **ayuda a elegir** la solución más adecuada en cada situación.

En muchos casos, una buena elección **ahorra** tiempo y dinero.

En algunos casos, una buena elección marca la diferencia entre **poder** resolver un problema y **no poder** hacerlo.



¿A quién puede interesar este curso?

- Interés en diseñar algoritmos para resolver nuevos problemas o algoritmos mejores a los triviales para resolver viejos problemas, y
 $O(n^2) \rightarrow \text{¿podemos?} \rightarrow O(n \log n)$
- Curiosidad por conocer, por ejemplo, cómo se resuelven los siguientes problemas:
 - ¿Cómo multiplicar dos números enteros de forma más eficiente que con el algoritmo clásico?
 - ¿Cómo se compactan ficheros con un algoritmo voraz?
 - ¿Cómo se consiguen implementaciones muy eficientes de la multiplicación y potenciación de enteros muy grandes para el algoritmo RSA?



¿A quién puede interesar este curso?

- ¿Por qué funcionan algoritmos vistos en Matemática Discreta como el de Dijkstra, Prim o Kruskal? ¿Podemos mejorar la forma de programarlos?
- ¿Podemos hacer mejores búsquedas suponiendo que se conocen las probabilidades de búsqueda de cada clave?
- ¿Alguna forma de resolver el problema del viajante de comercio?
- ¿Cómo se compara el ADN de un Homo sapiens y de un Homo neanderthalensis?

Bibliografía Básica



- CLRS09** T.H. Cormen, C.E. Leiserson, R.L. Rivest, C. Stein. Introduction to Algorithms (4th edition), The MIT Press, 2009. <https://mitpress.ublish.com/book/introduction-to-algorithms-4>
- DPV08** S. Dasgupta, C. Papadimitriou, U. Vazirani. Algorithms, McGrawHill, 2008.
- BB97** G. Brassard, P. Bratley. Fundamentos de Algoritmia, Prentice Hall, 1997.



Bibliografía Complementaria

- HSR08 E. Horowitz, S. Sahni, and S. Rajasekaran. Computer algorithms, 2nd ed., Silicon Press, 2008.
- KT05 J. Kleinberg, E. Tardos. Algorithm Design, Addison-Wesley, 2005.
- PG02 I. Parberry and W. Gasarch. Problems on Algorithms (2nd edition), free book, 2002.
<https://ianparberry.com/books/free/>

Además: transparencias y otro material adicional disponibles en la web de la asignatura, hojas de ejercicios, enlaces de Internet (ejemplo: <http://jeffe.cs.illinois.edu/teaching/algorithms/>), etc.



Chuleta (recordatorio)

Coste de algoritmos recursivos

El coste $T(n)$ de una función para datos de tamaño n se define en función del coste $T(m)$ de las llamadas recursivas para otros tamaños m (menores que n)

Teorema (Master Theorem):

$$T_1(n) = \begin{cases} f(n) & 0 \leq n < c \\ a \cdot T_1(n/c) + b \cdot n^k & c \leq n \end{cases} \Rightarrow T_1(n) \in \begin{cases} \Theta(n^k) & a < 1 \\ \Theta(n^{k+1}) & a = 1 \\ \Theta(a^{n/c}) & a > 1 \end{cases}$$

$$T_2(n) = \begin{cases} g(n) & 0 \leq n < c \\ a \cdot T_2(n/c) + b \cdot n^k & c \leq n \end{cases} \Rightarrow T_2(n) \in \begin{cases} \Theta(n^k) & a < c^k \\ \Theta(n^k \cdot \log n) & a = c^k \\ \Theta(n^{\log_c a}) & a > c^k \end{cases}$$